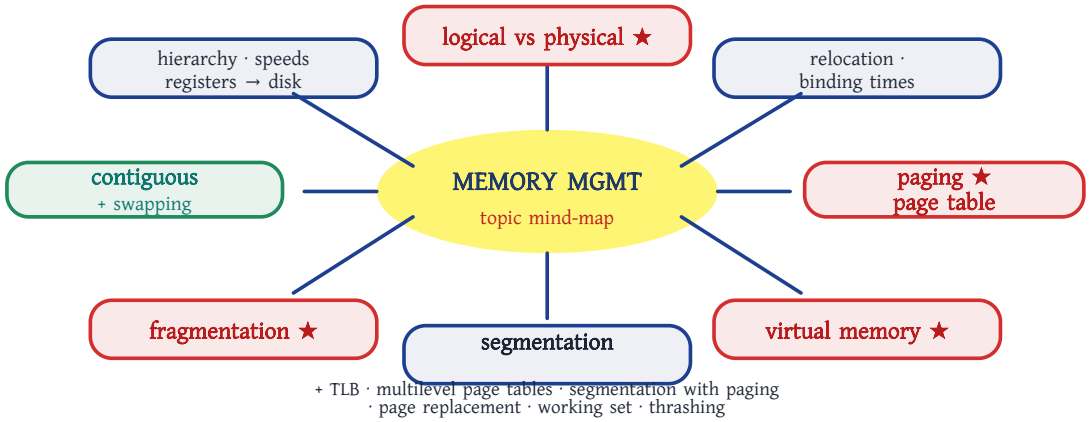
Memory Management

Paging · segmentation · the TLB · page faults · replacement · exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = the scheme the first questions come from

Syllabus coverage

The syllabus line is "**Memory management: swapping, paging, segmentation, virtual memory**". Scheduling and IPC are in the Process lesson; disk structure is in the Input/Output lesson.

High-yield list

① Paging kills external frag, not internal
② p = LA div P · d = LA mod P
③ PA = f·P + d · offset = $\log_2 P$ bits
④ TLB hit = 1 access, miss = 2
⑤ 32-bit + 4 KB ⇒ 2^{20} entries = 4 MB
⑥ Seg: PA = Base[s] + d, d < Limit[s]
⑦ Variable segments ⇒ external frag
⑧ Fault: trap, test, take, read, PTE, retry
⑨ EAT = (1 − p)m + p·f
⑩ Thrashing ⇒ admit fewer processes

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

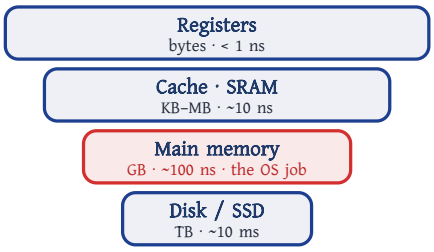
Contents

✓ Why memory management? … p. 2
✓ Fragmentation & swapping … p. 3
✓ Paging & translation … p. 4
✓ TLB, multilevel, PTE bits … p. 5
✓ Segmentation … p. 6
✓ Virtual memory & page fault … p. 7
✓ Page replacement … p. 8
✓ Working set & thrashing … p. 9
✓ Night-before p. 10 · Practice p. 11

What the whole lesson is for: Every idea here — the fixed-size page, the offset that is copied rather than moved, the dirty bit that decides whether a disk write happens — keeps three promises at once:
the program gets a flat address space, **two programs never overlap**, and
the OS can move anything at any moment.

1 The Hierarchy, Binding and Addresses

Definition: **Memory management** controls how memory is **allocated to and reclaimed from** processes. Its two goals: **maximise memory utilisation** and **minimise the time spent swapping**. It also supplies **abstraction** (memory looks like a flat list of addresses), **protection** (no process touches another's memory) and **sharing**.



up = faster and smaller. The OS manages the **middle band only**

Level	Size	Access	Managed by
Registers	bytes	< 1 ns	compiler
Cache (SRAM)	KB – MB	1 – 100 ns	hardware
Main memory (DRAM)	GB	50 – 100 ns	**the OS — this lesson**
Secondary storage	TB	50 µs – 10 ms	file system + swap

Binding time	The address is fixed	If the process moves
Compile time	while compiling; the structure is already known — absolute addresses	**must be recompiled**
Load time	when it is loaded into memory — relocatable code, base + limit set at load	just reload it
Execution time ★	on **every reference** — the MMU translates, using base/limit or a page table	**nothing — true dynamic relocation**

Logical vs physical: The **logical (relative)** address is what the program generates — counted from **0**, unrelated to where the program sits. The **physical (absolute)** address is the address in the real RAM unit; **only the memory hardware sees it**. Translating one into the other is **relocation**: because it happens while the program runs, the OS may move a process at any moment.

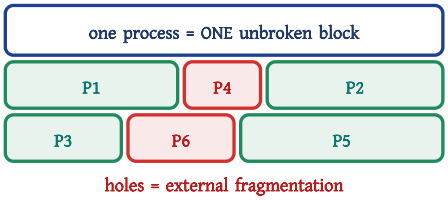
Base + limit: **Relocation (base) register** = where the partition begins; **limit register** = its size. Every logical address a is checked against the limit, then **physical = logical + base**. If $a \geq$ limit the CPU traps and aborts the process.

Memory trick: "**Compile = frozen, load = pinned, run = live.**" Only **execution-time** binding lets a program move without being recompiled — which is why every real machine has an MMU. Hold on to this: **the program sees logical, only the chips see physical.**

Exam trap: (i) Binding at **load** time still needs relocatable code; **execution** time is what makes swapping and compaction possible. (ii) Absolute addresses mean **recompiling** for every new location. (iii) Base + limit belong to **contiguous** memory, the page table to **paging**.

2 Fragmentation, Contiguous Allocation & Swapping

Definition: **Contiguous allocation** gives each process **one unbroken block** of memory, guarded by a **base (relocation)** register and a **limit** register. **Fragmentation** is free memory that cannot be used: **internal** = space **inside** a block already handed out, **external** = free space **between** blocks, not contiguous enough for the next request. Internal waste can never be recovered; external waste is cured only by **compaction**.



only the biggest contiguous run counts, never the sum of the free memory

compaction closes the holes; internal waste it can never touch

Strategy	Picks the hole that is...	Verdict
First fit	first big enough, from the start	fast; slivers left at the **end**
Best fit	smallest hole that still fits	**worst** leftover slivers
Worst fit	largest hole available	destroys the big block too soon
Next fit	first that fits, resuming where it stopped	spreads the damage evenly

Exam trap: (i) "**Which suffers external fragmentation?**" → contiguous and segmentation, **never paging**; "which internal?" → paging and **fixed** partitions. (ii) Compaction needs relocation hardware, i.e. **execution-time binding**. (iii) A swap can double the free space on paper and still leave no block big enough.

Swapping: **Swapping** moves an **entire process** out to the swap device on disk and later brings it back, which lets the system run **more processes than physical memory** and is a convenient moment to compact. Its whole cost is disk I/O. Swapping ≠ paging: swapping moves a **whole process** on the OS's initiative; paging moves **one page** when the program itself references it, and is invisible to the process.

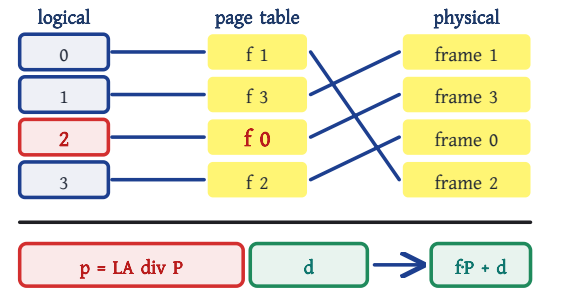
Worked example ✍ what does a swap cost?: A disk transfers at 10 MB/s with ~10 ms seek. **(a)** A 1 MB process out and back = **200 ms + 20 ms = 220 ms**. **(b)** A 33 MB process swapped twice each way = 132 MB ÷ 10 = **13.2 s**. **(c)** One 4 KB page instead = **0.4 ms** — **550× cheaper**. **(d)** Paging, which moves only the pages actually needed, is therefore what keeps a machine responsive.

Memory trick: "**Inside = internal, outside = external.**" Compaction can only fix the outside kind — the inside kind is baked into the size you handed over. Contiguous allocation needs one whole free hole; **paging never does**, which is exactly why paging exists.

In practice: Demand paging has almost killed classic swapping. A machine with plenty of RAM never swaps; when it does, the kernel pages out cold anonymous memory, and the swap file is a last resort, not a workspace.

3 Paging: Pages, Frames, Page Table, Translation

Definition: **Paging** divides both memories into **fixed-size blocks**: a process's logical memory into **pages**, physical memory into **frames**, **page = frame in size**. A process may then occupy **any** frames in any order — contiguity is gone. The **page table** is an array of **page-table entries (PTEs)**, one per page, each holding a frame number plus control bits.



e offset d is copied unchanged — only the page number is replaced
page 2 lives in frame 0: the pages sit in no order at all

one frame per page, in any order — that sentence is the whole idea of paging

Term	Meaning
Page	a fixed-size block of one process's **logical** memory
Frame	a fixed-size block of **physical** memory, same size as a page
Page table	per-process array of PTEs; it *is* the process's map
PTE	frame number + valid / protection / dirty / reference
Free-frame list	frames nobody is using, available at once

Exam trap: (i) The offset is copied **unchanged** — never add it to anything but f·P. (ii) **Paging removes external fragmentation only**; the last page is usually part-full, wasting on average **half a page** per process. (iii) Frames are the unit of allocation, so nothing inside a frame can be shared.

The four formulas — every paging question is one of them:

page number $p = \lfloor LA \div P \rfloor = LA \text{ div } P$

offset $d = LA \bmod P$

physical address $PA = f \cdot P + d$

pages in a process $\text{size} \div P$

If $P = 2^n$ bytes the offset needs exactly $n = \log_2 P$ **bits** and the page number takes the rest of the address. 4 KB = 2^{12}, so a 32-bit address splits into **20-bit p + 12-bit d**.

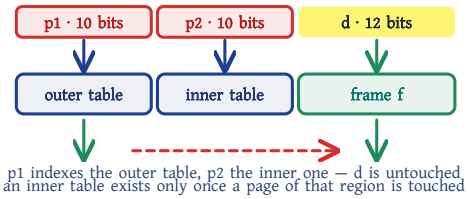
Worked example ✍ logical 3000, page 2 → frame 5: P = 1 KB = 1024. **(a)** p = 3000 div 1024 = **2** (2 × 1024 = 2048; 3 × 1024 = 3072 > 3000). **(b)** d = 3000 mod 1024 = 3000 − 2048 = **952**. **(c)** PTE[2] = **5**. **(d)** PA = 5 × 1024 + 952 = 5120 + 952 = **6072**. **(e)** Check: 5 × 1024 = 5120 ≤ 6072 < 6144 ✓. **(f)** The offset travels from the logical address to the physical one completely untouched.

Memory trick: "**Page in, frame out.**" The page number is an **index** into the page table; the offset is a **position** inside the block. Paging swaps the index and keeps the position.

IT link: Every x86-64 and ARM64 CPU walks exactly this table and caches recent PTEs in a **TLB** — the next sheet.

4 The TLB, Multilevel Page Tables & PTE Bits

Definition: Consulting the page table on every reference would make each access cost **two** trips. The **translation lookaside buffer (TLB)** is a small, very fast **associative cache of recent PTEs** inside the MMU. On a **hit** the frame number comes from the TLB and the data is fetched in **one** further access; on a **miss** the page table is read from memory and the TLB is refilled.



a 32-bit address with 4 KB pages: 20 page-number bits, split 10 + 10

TLB cost:

Let **t** = TLB lookup, **m** = one memory access, **h** = hit ratio.

EAT = h·(t + m) + (1 − h)·(t + 2m)

Scheme, 32-bit + 4 KB pages	Size
Single-level	2^{20} entries × 4 B = **4 MB per process**
Two-level (10 + 10)	outer 2^{10} × 4 B = **4 KB**, plus 4 KB per inner table in use
What you save	a process using 32 pages holds **4 KB + 128 B**, not 4 MB

Worked example ✍ TLB timing: A TLB lookup takes **20 ns**, one memory access **100 ns**, hit ratio **80 %**. **(a)** Hit = 20 + 100 = **120 ns**. **(b)** Miss = 20 + 100 (page table) + 100 (data) = **220 ns**. **(c)** EAT = 0.8 × 120 + 0.2 × 220 = 96 + 44 = **140 ns**. **(d)** Without a TLB it would be 200 ns. **(e)** Multilevel tables do not change this arithmetic — they only make the *miss* shorter, because a missing outer entry ends the walk at once.

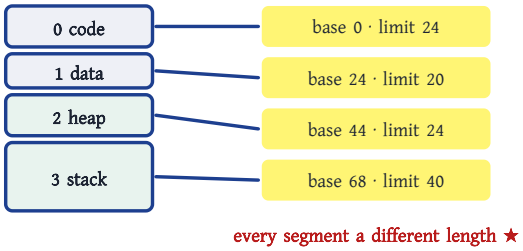
PTE bit	1 means	0 means	Read by
Valid	page is in a frame now	not in memory → **page fault**	MMU, every ref
Protection	read / write / execute ok	denied → fault	MMU, every ref
Dirty	written to since loading	unchanged → no write-back	replacement
Reference	touched recently	not touched	**clock**, LRU

Exam trap: (i) The TLB is **hardware**, not a table in memory — usually **associative memory** searched by content. (ii) Dirty = "changed", reference = "looked at"; swapping the two words loses the answer. (iii) A page can be **valid and never written** — validity is about presence, not modification.

Memory trick: "**Hit once, miss twice.**" And "**D for Dirty, R for Referenced** — dirty decides *whether to write back*, reference decides *who goes out next.*" A TLB is a cache of the page table, so a huge table is still cheap while the working set of pages is small.

5 Segmentation, and Segmentation with Paging

Definition: **Segmentation** divides memory into **variable-length logical units** — code, data, stack, heap — instead of equal blocks. A logical address is the pair **(segment number, offset)**. The **segment table** holds, per segment, a **base** (starting physical address) and a **limit** (length), plus protection bits, so **PA = Base[s] + d** whenever **d < Limit[s]**; otherwise a **segmentation fault**.



the variable-length partition problem again. Worked: segment 2 has base 44, limit 24, so (2, 23) → 67 ✓ but (2, 24) → fault, because 24 ≥ limit

Point	Paging	Segmentation
Block size	fixed	**variable**
Address	(p, d)	(s, d)
Table entry	frame number + bits	base + limit + bits
Fragmentation	internal only	**external** only
Grows with data?	no	yes — heap, stack
Protection / sharing	per page, awkward	natural, per segment

Exam trap: (i) Segmentation gives **external** and paging gives **internal** fragmentation — opposite diseases. (ii) A segment number is **not** a page number; segments have no fixed size, so an address cannot be split into bits. (iii) Sharing a whole segment is natural; sharing **half** a page is not.

Segmentation with paging (segmented paging):

Every segment is **again** divided into fixed-size pages and **each segment gets its own page table**; the segment-table entry now holds the **base address of that segment's page table** instead of a base+limit pair. A logical address **(s, p, d)** is translated in **three** steps — segment table → the page table of segment *s* → the frame of page *p* inside it — giving PA = f·P + d. The segment table itself can be small enough to sit in registers, so only the two memory hops cost anything.

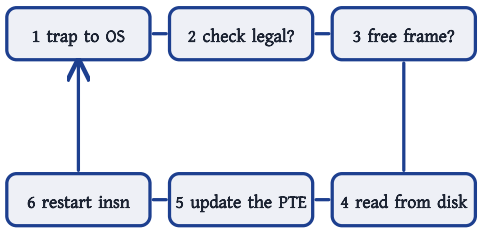
Memory trick: "**Base adds, offset selects, limit refuses.**" In paging the frame number **replaces** the page number and the offset is copied; in segmentation the base is **added** to the offset.

What real machines do: x86 real mode is pure segmentation; x86 protected mode is **segmented paging** — a selector picks a descriptor, which supplies the page-table base, then ordinary paging runs. ARM64 and Linux dropped segmentation entirely.

Exam trap: The **limit** is a **length**, not an end address: with base 44 and limit 24 the last legal offset is **23**, because d < Limit. A **segmentation fault** and a **page fault** are different traps with different causes.

6 Virtual Memory, Demand Paging & the Page Fault

Definition: **Virtual memory** lets a process be **executed while only part of it is in physical memory**: the program works with its own **virtual address space**, which may be larger than RAM, and the OS keeps the unused part on the back store. **Demand paging** is the usual implementation — **load a page only when it is first referenced**, also called **lazy loading**. Each PTE carries a **valid–invalid bit**: valid = present in memory, invalid = not present.



the six steps, in the order an examiner wants them
step 6 is not optional

the loop closes because the faulting instruction is re-executed from the beginning

Servicing a page fault — six steps, in order:

1. The MMU finds **valid = 0** and **traps to the OS**; the process is blocked.
2. The OS checks the reference is **legal**. Illegal → **abort the process**.
3. Take a **free frame**; if there is none, run a **page-replacement algorithm** and write the victim out first if its **dirty bit** is set.
4. Schedule a **disk read** of the required page into that frame; the process waits.
5. When the read completes, **update the page table**: frame number, **valid = 1**, dirty = 0.
6. **Restart** the instruction that caused the fault.

Worked example ✍ effective access time: m = 100 ns per access, f = 10 ms per fault, p = fault rate. **EAT = (1 − p)·m + p·f.** **(a)** $p = 10^{-6}$: EAT ≈ $100 + 10^{-6} \times 10^{7}$ = **110 ns.** **(b)** For the fault overhead to stay under one memory access, $p \cdot f < m$, so $p < 100\text{ ns} \div 10^{7}\text{ ns} = \mathbf{10^{-5}}$. **(c)** $p = 10^{-3}$ gives about **10 µs** — a hundred times slower. **(d)** This is why a fault rate above roughly one in 100 000 is fatal, and why the working-set model exists.

Exam trap: (i) A page fault is **not** an error — it is the normal way a page arrives. (ii) The dirty bit is checked **before** the disk read, and only a dirty victim is written back. (iii) "Invalid" means **not in memory**, not "broken". (iv) Virtual memory lets **many** processes share the RAM, not just one process exceed it.

Memory trick: "**Trap, test, take, read, write the PTE, retry.**" Six words, always in that order. And the last word is the giveaway: **if you never restart the instruction, the process stays dead for ever**.

In practice: Reference counting and garbage collection are the same idea used for a different purpose: map a known-good image into memory, fault it in lazily, and page it out untouched. Linux calls the second case a **minor fault** — no disk I/O, just fill the PTE.

7 Page Replacement: FIFO, Optimal, LRU, Clock

Definition: **Page replacement** runs when a process needs a frame and none is free: the OS picks a resident page, writes it out if dirty, and reuses its frame. **Page-fault rate** PFR = page faults ÷ total references. **No algorithm avoids the first reference to a page**, and **more frames is not always fewer faults — Belady's anomaly**.

Algorithm	Evicts the page that is...	Realisable?	Verdict
FIFO	loaded longest ago, regardless of use	yes — a queue	simple; suffers Belady's anomaly
Optimal (MIN)	needed **furthest in the future**	no — needs future knowledge	**the benchmark, not a real choice**
LRU	used **longest ago**	exactly; practically approximated	good; no Belady anomaly
Clock ★	found with a reference bit = 0 on a sweep	yes — cheap, no extra bit	LRU's idea at LRU's cost

Worked trace ✍️ string 1 2 3 4 1 2 5 1 2 3 4 5, three frames, all empty — F = page fault, "hit" = already in a frame:

ref	page	FIFO frames	F/L	LRU frames	F/L	Optimal frames	F/L
1	1	1 – –	F	1 – –	F	1 – –	F
2	2	1 2 –	F	1 2 –	F	1 2 –	F
3	3	1 2 3	F	1 2 3	F	1 2 3	F
4	4	4 2 3	F	2 3 4	F	1 2 4	F
5	1	4 1 3	F	3 4 1	F	1 2 4	hit
6	2	4 1 2	F	4 1 2	F	1 2 4	hit
7	5	5 1 2	F	1 2 5	F	1 2 5	F
8	1	5 1 2	hit	2 5 1	hit	1 2 5	hit
9	2	5 1 2	hit	5 1 2	hit	1 2 5	hit
10	3	5 2 3	F	1 2 3	F	3 2 5	F
11	4	5 3 4	F	2 3 4	F	3 4 5	F
12	5	5 3 4	hit	3 4 5	F	3 4 5	hit
faults			**9**		**10**		**7**

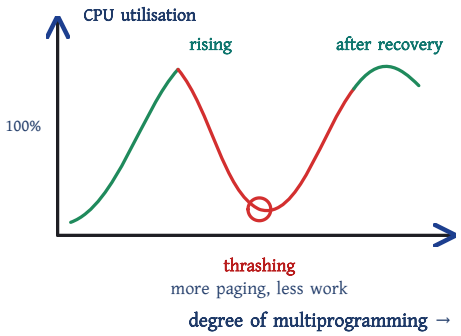
Exam trap: Belady's anomaly: with the **same** string and **four** frames FIFO gives **10 faults — more than the 9 it gives with three**. So "more frames means fewer faults" is **false** for FIFO and true for LRU and Optimal. Also, the first reference to each distinct page always faults, so **at least 5 faults are unavoidable** here.

Memory trick: **"MIN is the ceiling, FIFO is the floor."** Exact LRU needs a timestamp or a stack per frame, so real systems use **clock**, which just walks the frames and clears reference bits. FIFO forgets use; LRU remembers it; **clock remembers it cheaply**.

IT link: Linux approximates LRU with **multi-gen LRU**: page-table entries carry young/old bit pairs, so no scan is needed. Windows and macOS use a variant of clock. The exam name for all of them is still "an LRU approximation".

8 Working Set, Thrashing & Its Control

Definition: The **working set** of process *i* at time *t* is **$WSS_i(t)$** = the pages it referenced in the last **Δ time units** (Δ ≈ a few milliseconds, enough to cover its normal burstiness). Total demand is **$D(t) = \Sigma WSS_i(t)$**. If **D(t) > the number of free frames**, pages are thrown out only to be wanted again — **thrashing**.



adding one more process helps until the frames run out — then it hurts badly

Two ways to stop thrashing:

1. **Working-set policy.** Keep each process allocated exactly its WSS. On a fault, if demand exceeds what is free, **evict pages from a process whose WSS is shrinking**, or suspend a whole process. Freed frames go to the process whose WSS is growing.

2. **Page-fault-frequency (PFF) control.** Measure each process's PFR over a window. If PFR > **upper bound**, give it a free frame; if PFR < **lower bound**, take one back. Direct, and needs no Δ.

Symptom	Meaning
PFR climbing	too few frames for the working set
CPU utilisation **falling**	the busy process is spending its time in I/O — **thrashing**
Disk writes climbing	dirty pages written out again and again

Worked example ✍️ does it thrash?: Four frames are free. **(a)** WSS(P1) = 6, WSS(P2) = 2, WSS(P3) = 3, so D = 11 > 4 → **thrashing**: suspend P3 until frames free up. **(b)** P3's working set then collapses to 1 page; D = 6 + 2 + 1 = 9, still > 4 → suspend P2, whose WSS has fallen to 0 → D = 7. **(c)** Reduce once more: D = 6 ≤ available frames → **stable**, utilisation climbs again. **(d)** PFF reading: P1 runs at 0.3 faults per reference, far above the upper bound — it is the culprit.

Exam trap: (i) **Thrashing ≠ deadlock**: the system is busy, not stuck — pages fly in and out. (ii) Thrashing is **high CPU utilisation → low**; in a deadlock nothing moves at all. (iii) Adding processes always helps **up to** the point D > free frames, and hurts after it.

Memory trick: "**Demand over supply = thrashing.**" If the working sets will not fit, the OS either **suspends a process** or **gives frames to the greediest one**. And the cure that always works is the boring one: **admit fewer processes**.

Night-Before-the-Exam Sheet

Fact sheet:

Binding: compile / load / execution

Execution time = **relocation**

Logical = from 0, **relative**

physical = in the RAM unit

Base + limit ⇒ contiguous

Internal = inside a block

External = between blocks

Compaction fixes external only

Page = frame size, **fixed**

p = LA **div** P · d = LA **mod** P

PA = f·P + d

$P = 2^n$ ⇒ offset = **n bits**

32-bit + 4 KB ⇒ 2^{20} entries

Single-level PT = **4 MB**

TLB hit = 1 access, miss = 2

PTE: valid, protect, dirty, ref

Dirty = changed · Ref = looked at

Segment: PA = **Base[s] + d**

d ≥ Limit[s] ⇒ fault

Seg + paging ⇒ (s, p, d)

Segments **grow** ⇒ external frag

VM: only part of a process in RAM

Demand paging = **lazy**

Fault: trap→test→take→read→PTE→retry

EAT = (1 − p)m + p·f

FIFO may show **Belady**

LRU and MIN never do

Clock = a cheap LRU

WSS = pages in the last Δ

D(t) > free frames ⇒ **thrashing**

Fix: suspend, or PFF control

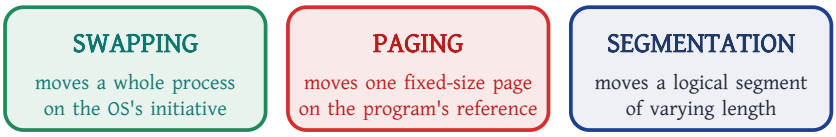
PFR = faults ÷ references

Top 12 one-liners for MCQs

1. Physical memory = **frames**, logical = **pages**
2. Offset = LA **mod** P, copied **unchanged**
3. $P = 2^n$ ⇒ the offset is exactly **n bits**
4. P 1 KB, page 2 → frame 5, LA 3000 ⇒ **6072**
5. TLB hit = **one** memory access
6. TLB miss = **two** memory accesses
7. 32-bit + 4 KB page ⇒ 2^{20} entries
8. 4-byte PTE ⇒ **4 MB** single-level table
9. Segment fault when **d ≥ Limit[s]**
10. Paging ⇒ **internal** fragmentation
11. VM ⇒ **demand paging**, valid–invalid bit
12. Thrashing ⇒ **fewer processes**

Top traps: adding the offset to the base **and** to the frame · paging said to remove **all** fragmentation · FIFO said to be immune from Belady · the dirty bit confused with the reference bit · segmentation said to have fixed-size blocks · a page fault treated as an error · "more frames always means fewer faults" · virtual memory said to let **one** process exceed RAM.

All tricks together: "Compile = frozen, load = pinned, run = live" · "the program sees logical, only the chips see physical" · "inside = internal, outside = external" · "page in, frame out" · "hit once, miss twice" · "D for Dirty, R for Referenced" · "base adds, offset selects, limit refuses" · "trap, test, take, read, write the PTE, retry" · "MIN is the ceiling, FIFO is the floor" · "demand over supply = thrashing".



"what moves, how big, and who decided"

the three words that separate the three schemes — this question is asked constantly

✓ Practice MCQs (exam pattern, 10 marks)

1. Paging removes external fragmentation, but still suffers:
(a) starvation (b) race conditions
(c) convoy effect (d) internal fragmentation

2. More time swapping pages than executing means:
(a) thrashing (b) starvation
(c) deadlock (d) fragmentation

3. Page 1 KB, page 2 → frame 5, logical 3000. Physical:
(a) 5072 (b) 3952
(c) 6072 (d) 7096

4. 32-bit address, 4 KB pages. Entries in one table:
(a) 2^{12} (b) 2^{20}
(c) 2^{32} (d) 2^{10}

5. A PTE's **dirty bit** is set when the page:
(a) is loaded in (b) is first referenced
(c) is modified (d) is swapped out

6. TLB 20 ns, memory 100 ns, hit ratio 80 %. Access time:
(a) 120 ns (b) 140 ns
(c) 150 ns (d) 220 ns

7. In segmentation an offset **≥ the limit** causes:
(a) a TLB miss (b) a page fault
(c) compaction (d) a segmentation fault

8. With a 4 KB page size the offset field needs:
(a) 4 bits (b) 12 bits
(c) 20 bits (d) 32 bits

9. Demand paging loads a page:
(a) at load time (b) in advance
(c) never (d) on first reference

10. Segmentation with paging addresses memory as:
(a) (p, d) (b) (s, d)
(c) (s, p, d) (d) a linear address

Answer key → 1-d · 2-a · 3-c · 4-b · 5-c · 6-b · 7-d · 8-b · 9-d · 10-c

Written question: What is virtual memory? Explain demand paging and the steps the operating system takes to service a page fault. 10

1. **Virtual memory** — a technique that lets a process execute when only part of it is in physical memory; the program works with its own **virtual address space**, which may be larger than RAM, and the OS keeps the unused pages on the back store. 3
2. **Demand paging** — pages are loaded **on demand**, only when first referenced (lazy loading). Each PTE carries a **valid–invalid bit**: valid = the page is in memory, invalid = it is not. 3
3. **The six steps, in order** — (1) the MMU finds valid = 0 and **traps to the OS**, blocking the process; (2) the OS checks the reference is **legal**, aborting the process if not; (3) it takes a **free frame**, or runs a **page-replacement algorithm** to choose a victim and **writes the victim out first if its dirty bit is set**; (4) it **schedules a disk read** of the page into that frame; (5) when the read completes it **updates the page table** — frame number, valid = 1, dirty = 0; (6) it **restarts the instruction** that caused the fault. 4

Before the exam ✓

- ✓ Can split an address into page number and offset
- ✓ Can compute a physical address from a PTE
- ✓ Can trace a reference string under FIFO and LRU
- ✓ Can recite the six page-fault steps in order